

# When a Machine Learns to Read

---

Introduction to Natural Language Processing · AI and Machine Learning · Week 2, Day 4

# Where we left off

We taught machines to see. Now we teach them to read. Today's path:

## Text to numbers

Turn words into something a model can use.

## Meaning

Embeddings place similar words close together.

## Attention

Let every word look at the others that matter.

**From counting words, to capturing meaning, to the idea that powers modern AI: attention.**

We will spend real time on attention, the engine inside every modern language model.

## Part 1

# What Is Natural Language Processing?

---

# What is NLP?



# What is NLP?

NLP: building systems that read, understand and generate human language.

1

**Autocomplete**

2

**Translation**

3

**Voice assistants**

4

**Spam filters**

In every case the input is language, and the output is a useful decision or response.

# Why language is hard for computers?



# Why language is hard for computers

## Ambiguity

"bank": a river bank or a money bank?

## Order matters

"dog bites man" vs "man bites dog".

## Tone and context

Sarcasm: "great, another delay."

## Endless variety

Spelling, slang and many languages.



**Think** *what is a sentence whose meaning flips completely with one small change in wording?*

# NLP in your field?





# NLP in your field



## Environmental Science

Scan thousands of climate reports and papers for key findings.

## Materials Science

Pull properties and methods out of research papers and patents.

## Architecture

Search building codes and project specs written in plain language.

## Computer Engineering

Power chatbots, code assistants and automatic log analysis.

## Chemical Engineering

Mine safety reports and process logs for early warnings.

Most knowledge in every field is written as text. NLP helps you search, sort and summarise it.



## Part 2

# From Words to Numbers

---

# Step 1: split text into tokens



## Words or word pieces?

Modern models split rare or long words into subword pieces, so a small vocabulary can still spell any word, even one it never saw in training.

Tokenisation breaks text into pieces. Subword tokens keep the vocabulary small and handle new words.

# A first try: one-hot vectors

Give every vocabulary word its own slot. Mark its slot with a 1, the rest 0.

|   |    |       |         |          |     |
|---|----|-------|---------|----------|-----|
| I | am | happy | because | learning | NLP |
| 0 | 0  | 1     | 0       | 0        | 0   |

"happy" = [ 0, 0, 1, 0, 0, 0 ]

The trouble?

# A first try: one-hot vectors

Give every vocabulary word its own slot. Mark its slot with a 1, the rest 0.

| I | am | happy | because | learning | NLP |
|---|----|-------|---------|----------|-----|
| 0 | 0  | 1     | 0       | 0        | 0   |

"happy" = [ 0, 0, 1, 0, 0, 0 ]

**The trouble:** vectors are huge (one slot per word) and every word is equally far from every other. 'happy' is no closer to 'glad' than to 'NLP'.

One-hot vectors are simple but carry no meaning, and grow with the size of the vocabulary.

# Bag of Words counts the tokens

Text: "I am happy because I am learning NLP"



Bag of Words vector: [ 2, 2, 1, 1, 1, 1 ]

Count how often each word appears. The text becomes a vector, but word order is gone.

# TF-IDF: weight the words that matter

## Term frequency

How often a word appears in this text. Frequent here usually means important here.

## Inverse document frequency

Down-weight words common across all texts, like 'the' and 'is'. They carry little meaning.

**TF-IDF score = term frequency  $\times$  inverse document frequency. Rare, telling words score highest.**

A smarter Bag of Words: it highlights the words that actually distinguish one document from another.

# Counting still throws away the order

## Same words, opposite meaning

**"dog bites man"**

**"man bites dog"**

Identical counts. The model cannot tell them apart.

## No sense of similarity

"happy" and "glad" mean almost the same,  
but counting treats them as totally unrelated words.



**Think** *if a model ignores order and similarity, what important information about a sentence is lost?*





## Part 3

# Embeddings: Giving Words Meaning

---

# Embeddings turn words into meaning vectors

Each word becomes a short list of numbers learned from how it is used. Similar words get similar numbers.

|       |     |     |     |     |     |
|-------|-----|-----|-----|-----|-----|
| king  | 0.8 | 0.7 | 0.2 | 0.9 | ... |
| queen | 0.8 | 0.7 | 0.8 | 0.9 | ... |
| apple | 0.1 | 0.2 | 0.5 | 0.1 | ... |

**king** and **queen**  
share most numbers, so they sit close together.  
**apple** is far away in this space.

Instead of thousands of empty slots, a few dozen numbers capture what a word means.



# How embeddings are learned: Word2Vec

You learn a word from the company it keeps. Train a model to predict a word's neighbours.

Sliding window: "I am [ learning ] natural language"

*predict the surrounding words*

learning



am

natural

language

Words that predict the same neighbours end up with similar vectors. Meaning falls out of context.



# Measuring how close two words are

Cosine similarity: the similarity between two word vectors. Smaller value means similar meaning.



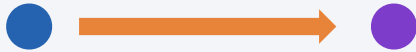
1.00 = same direction, identical meaning    0.00 = unrelated

This is how search and recommendation find related words, documents and products.

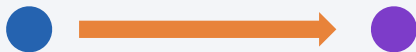


# Meaning becomes geometry

king                      queen



man                      woman



apple



banana



**Directions carry meaning.**

The step from man to woman is the same as king to queen.

**king - man + woman lands near queen.**

?

**Think** *which two words in your field are close in meaning, and what makes them similar?*



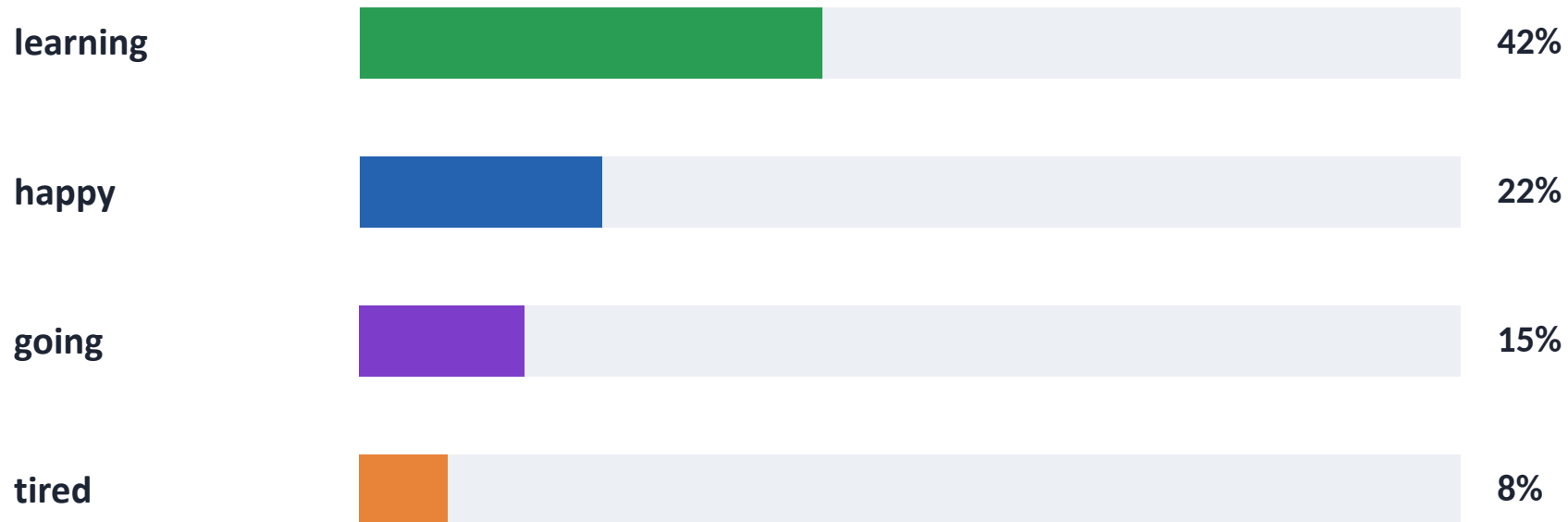
## Part 4

# Reading in Sequence

---

# From meaning to prediction: the next word

A language model reads "I am \_\_\_\_" and scores what comes next:



Trained on huge amounts of text, the model learns which words usually follow which. This is autocomplete.

# The old way: counting n-grams

Estimate the next word from how often short word sequences appeared in the training text.

Corpus: "I am happy because I am learning natural language"

$P(\text{am} \mid \text{I})$

$$2 / 2 = 1.0$$

$P(\text{happy} \mid \text{am})$

$$1 / 2 = 0.5$$

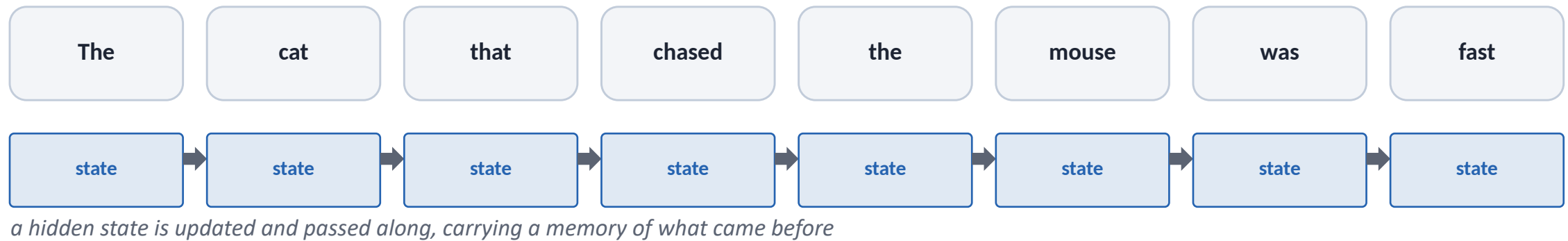
$P(\text{learning} \mid \text{am})$

$$1 / 2 = 0.5$$

Simple and fast, but a short window forgets far-back words and needs to have seen every phrase before.



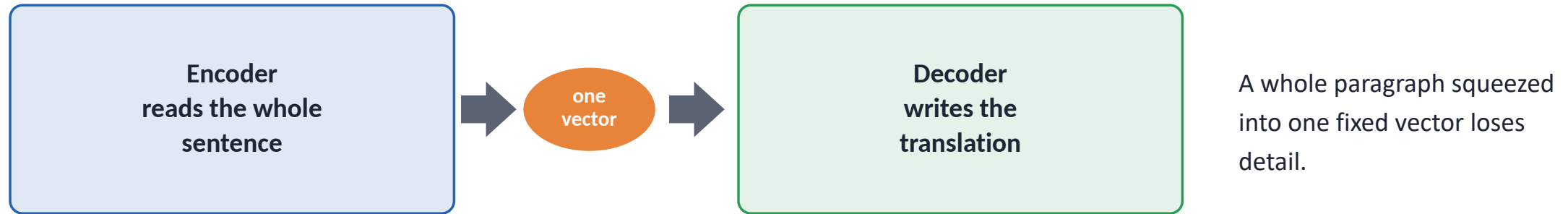
# Reading word by word: RNNs



**The weakness:** to pick "was" or "were" the model must recall "cat" from far back. Over long sentences the memory fades.

RNNs read in order and keep a running memory, but they struggle to link words that are far apart.

# The bottleneck that led to attention



**The fix: let the decoder look back at every word of the input, not just one summary vector. That is attention.**

Forcing all meaning through a single vector is the bottleneck attention was invented to remove.



## Part 5

# Attention: Letting Words Look at Each Other

---

# The big idea: which words should I look at?



The cat sat because **it** was sleepy



Reading "it", the model looks back at every word and puts most of its weight on "cat". That is attention.



# Attention is a weighted average

Step 1 score every word. Step 2 turn scores into weights that add to 1. Step 3 blend.

| word    | score | weight (after softmax) |
|---------|-------|------------------------|
| cat     | 4.0   | 0.70                   |
| sleepy  | 1.5   | 0.12                   |
| was     | 1.0   | 0.08                   |
| sat     | 0.5   | 0.05                   |
| The     | 0.2   | 0.03                   |
| because | 0.1   | 0.02                   |

## Result

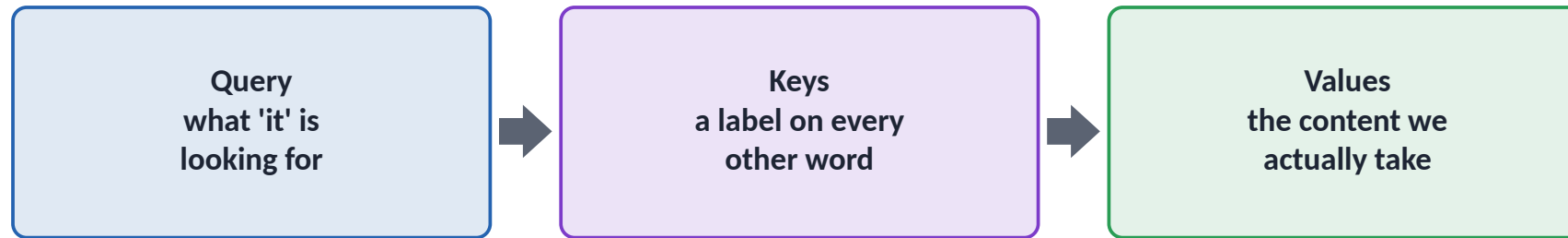
0.70·cat + 0.12·sleepy + ...  
mostly the meaning of 'cat'.

Softmax turns raw scores into weights that sum to 1. The output is just those words, blended by weight.



# Query, key and value

Think of a search. A query asks; each word offers a key to match; the value is its content.



$\text{score} = \text{query} \cdot \text{key}$  (a dot product: how well they match). Then softmax the scores into weights, and take that weighted blend of the values.

Every word produces its own query, key and value. The same three step recipe runs for all of them.



# Self-attention: every word looks at every word

query ↓

|        | The  | cat  | chased | the  | mouse |
|--------|------|------|--------|------|-------|
| The    | 0.55 | 0.20 | 0.10   | 0.10 | 0.05  |
| cat    | 0.10 | 0.55 | 0.15   | 0.05 | 0.15  |
| chased | 0.05 | 0.40 | 0.25   | 0.05 | 0.25  |
| the    | 0.10 | 0.10 | 0.10   | 0.55 | 0.15  |
| mouse  | 0.05 | 0.30 | 0.25   | 0.05 | 0.35  |

## Read a row.

The row for "chased" puts weight on "cat" and "mouse": its subject and object.

Darker cell = more attention. Every word builds its own blend in one shot.

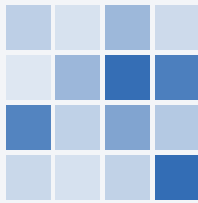
Self-attention computes a weight between every pair of words at once, so any two words can connect directly.



# Multi-head attention: several views at once

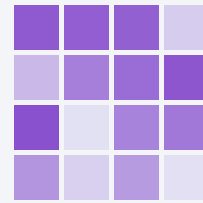
Run attention several times in parallel. Each head learns a different kind of relationship.

Head 1



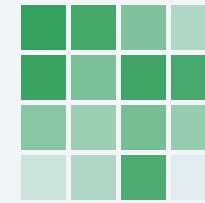
*who did what*

Head 2



*describing words*

Head 3



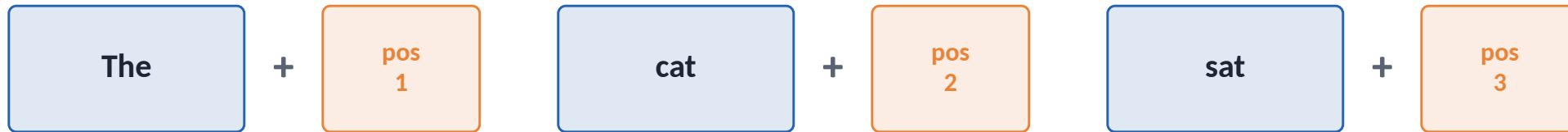
*what refers to what*

Combining many heads lets one layer track grammar, meaning and references all at the same time.



# Position matters: telling words their order

Attention alone treats a sentence as a bag with no order. So we stamp each word with its position.



Word meaning plus position goes into the model, so 'dog bites man' and 'man bites dog' look different again.

# Why attention changed everything



## Sees the whole sentence

Every word can look at every other word at once.



## Handles long range

Distant words link directly, nothing fades away.



## Trains in parallel

No waiting word by word, so it is fast on big data.



**Think** *in a long paragraph, why is it useful for the last word to look directly back at the first?*



## Part 6

# Transformers and LLMs

---



# Inside a Transformer block

Multi-head attention

Add and normalise

Feed forward network

Add and normalise

*stack this block many times*

**Attention** lets words share information.

**Feed forward** then thinks about each word on its own.

**Add and normalise** keep training stable, like skip connections in a CNN.

Stacking dozens of these blocks builds the depth that makes large models powerful.

A Transformer is mostly this one block, repeated. Attention plus feed forward, over and over.

# Encoders, decoders and cross-attention

## Encoder

Reads the whole input at once, both directions.  
Best for understanding: classification, search, question answering.

## Decoder

Generates one token at a time, looking back only at what it has written.  
Best for writing: chat, translation, code.

Translation uses both: the decoder cross-attends to the encoder, looking at the source while it writes.

# How a large language model is built?



# How a large language model is built

## 1 Pretrain

Read a huge slice of the internet and learn to predict the next token. Slow and costly, done once.

## 2 Fine-tune

Train a little more on your task or domain, with far less data.

## 3 Align

Use human feedback to make answers helpful and safe.

The same transfer-learning idea as vision: pretrain once on huge data, then adapt cheaply to your task.

# Two famous families: GPT and BERT

## GPT (a decoder)

Predicts the next token over and over.  
Great at generating text: chat, writing, code.

## BERT (an encoder)

Reads the whole sentence both ways at once.  
Great at understanding: classification, search, QA.

GPT, BERT and LLaMA are all Transformers, differing mainly in whether they read, write, or both.



# Talking to an LLM: prompting

You steer a pretrained model with words alone, no retraining. The prompt is the program.

## Zero-shot

Just ask. 'Summarise this report in three points.'

## Few-shot

Show a couple of examples first, then the new case.

## Step by step

Ask it to reason out loud before answering.

Because the model already learned language, clear instructions and examples are often all it needs.

# What NLP can do today

**Classification**

**Sentiment analysis**

**Named entities**

**Translation**

**Question  
answering**

**Summarisation**

**Code generation**

**Chat assistants**

All of these run on the same foundation: tokens, embeddings and attention, scaled up with lots of text.

# Powerful, but know the limits

## Confidently wrong

It can invent facts, called hallucination. Check important claims.

## Reflects its data

Bias in the text becomes bias in the answers.

## No real understanding

It models word patterns, it does not truly know the world.

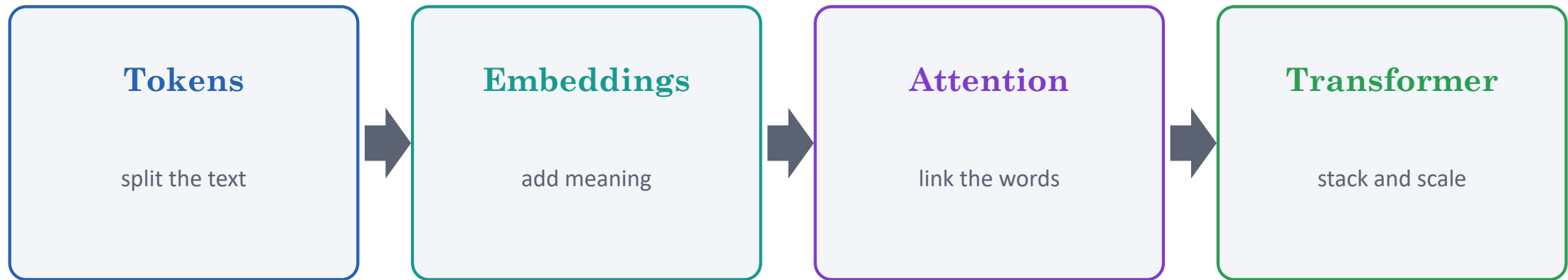
## Forgets and dates

Limited memory of long inputs, and a fixed training cut-off.

?

**Think** before trusting an LLM's answer in your field, what is one check you would always run?

# From text to a smart language model



Each step adds something: pieces, then meaning, then context, then depth. That is modern NLP.



# Key takeaways

1

**Text to numbers**

Tokenise, then represent words: one-hot, counts, or embeddings.

2

**Embeddings**

Similar words get similar vectors; directions carry meaning.

3

**Attention**

Score, soft-weight and blend: every word attends to the others.

4

**Self and multi-head**

All pairs connect at once, across several relationships.

5

**Transformers and LLMs**

Stack attention, pretrain on huge text, fine-tune and prompt.

# Questions?

---

You have now seen machines learn to see and to read. Next, we explore how they learn and create.